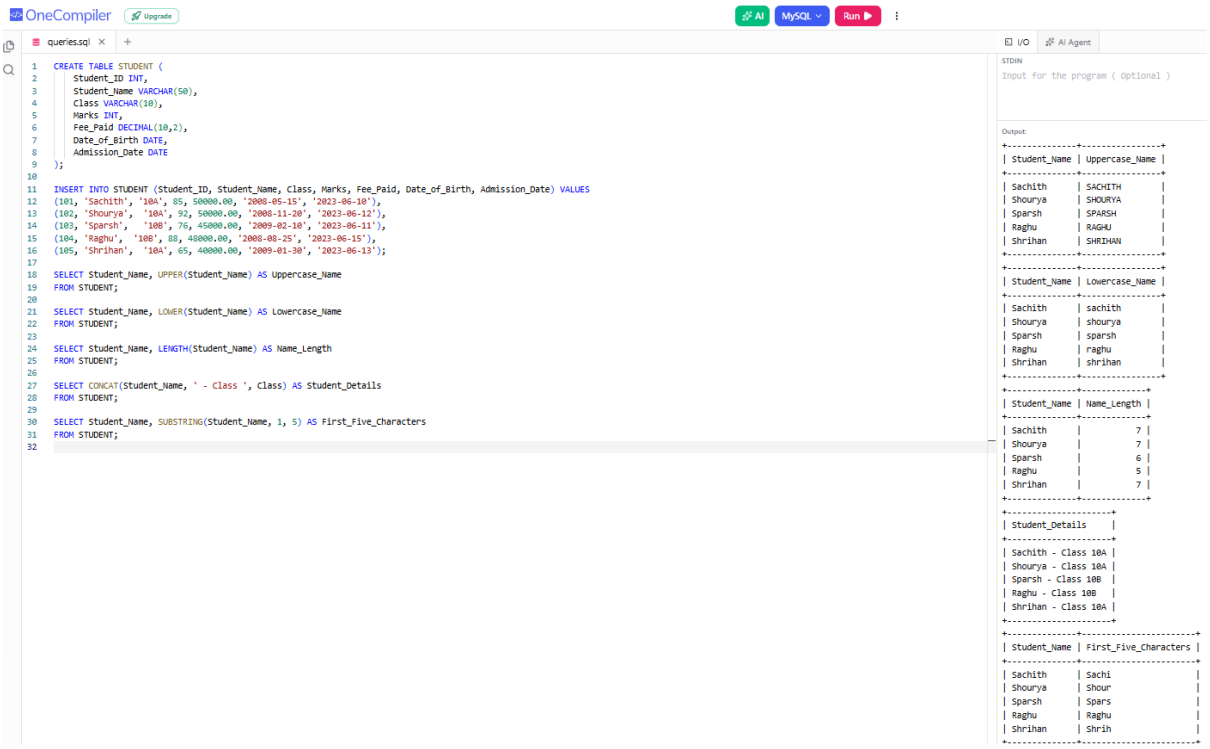
SQL Functions

Github Link - https://github.com/sachithn-27/Sql_Functions.git

1) String Functions



2) Numeric Functions –

OneCompiler Upgrade

queries.sql

```sql
CREATE TABLE STUDENT (
    Student_ID INT,
    Student_Name VARCHAR(50),
    Class VARCHAR(10),
    Marks INT,
    Fee_Paid DECIMAL(10,2),
    Date_of_Birth DATE,
    Admission_Date DATE
);

INSERT INTO STUDENT (Student_ID, Student_Name, Class, Marks, Fee_Paid, Date_of_Birth, Admission_Date) VALUES
(101, 'Sachith', '10A', 85, 50000.00, '2008-05-15', '2023-06-10'),
(102, 'Shourya',  '10A', 92, 50000.00, '2008-11-20', '2023-06-12'),
(103, 'Sparsh',   '10B', 76, 45000.00, '2009-02-10', '2023-06-11'),
(104, 'Raghu',  '10B', 88, 48000.00, '2008-08-25', '2023-06-15'),
(105, 'Shrihan',  '10A', 65, 40000.00, '2009-01-30', '2023-06-13');

SELECT
    Student_Name,
    Marks,
    ROUND(Marks / 10, 0) AS Rounded_Value,
    CEIL(Marks / 10) AS Ceiling_Value,
    FLOOR(Marks / 10) AS Floor_Value,
    ABS(Marks - 100) AS Absolute_Value,
    MOD(Student_ID, 2) AS Remainder,
    POWER(Marks, 2) AS Marks_Squared
FROM STUDENT;
```

Save

I/O AI Agent

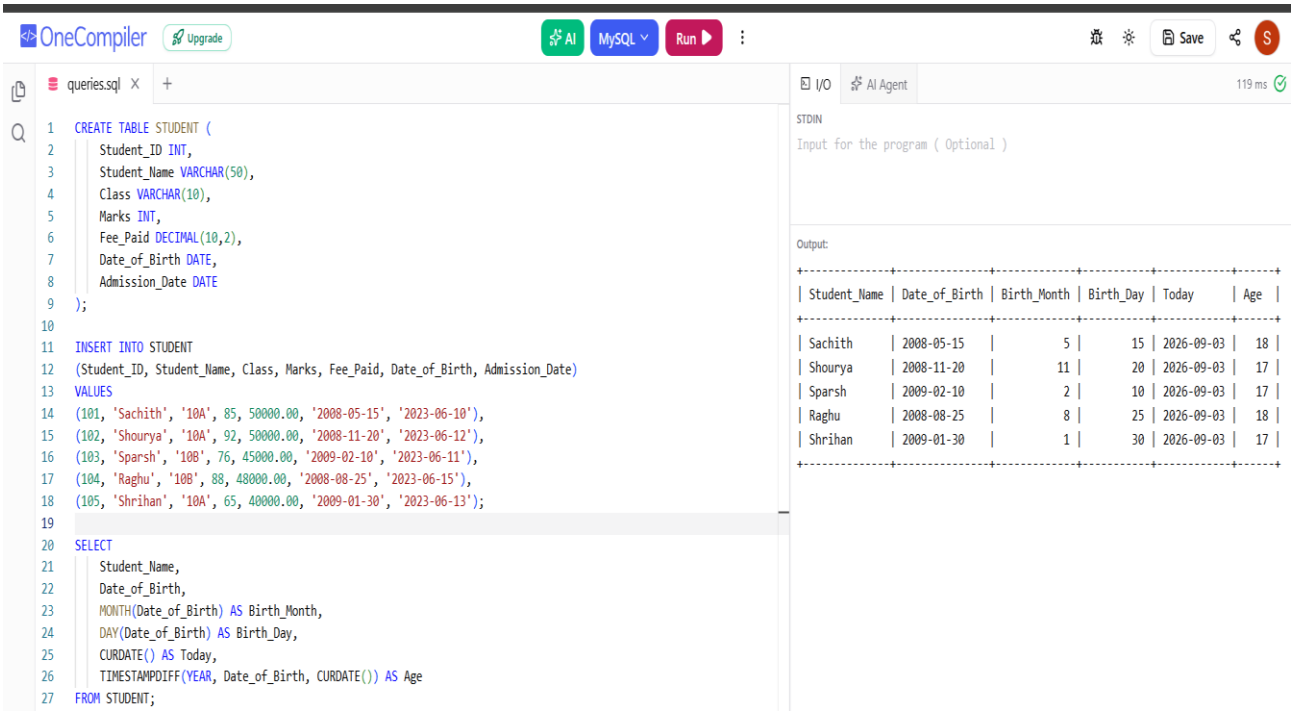
STDIN

Input for the program (Optional)

Output:

Student_Name	Marks	Rounded_Value	Ceiling_Value	Floor_Value	Absolute_Value	Remainder	Marks_Squared
Sachith	85	9	9	8	15	1	7225
Shourya	92	9	10	9	8	0	8464
Sparsh	76	8	8	7	24	1	5776
Raghu	88	9	9	8	12	0	7744
Shrihan	65	7	7	6	35	1	4225

3) Date Functions –




```
CREATE TABLE STUDENT (
    Student_ID INT,
    Student_Name VARCHAR(50),
    Class VARCHAR(10),
    Marks INT,
    Fee_Paid DECIMAL(10,2),
    Date_of_Birth DATE,
    Admission_Date DATE
);

INSERT INTO STUDENT
(Student_ID, Student_Name, Class, Marks, Fee_Paid, Date_of_Birth, Admission_Date)
VALUES
(101, 'Sachith', '10A', 85, 50000.00, '2008-05-15', '2023-06-10'),
(102, 'Shourya', '10A', 92, 50000.00, '2008-11-20', '2023-06-12'),
(103, 'Sparsh', '10B', 76, 45000.00, '2009-02-10', '2023-06-11'),
(104, 'Raghu', '10B', 88, 48000.00, '2008-08-25', '2023-06-15'),
(105, 'Shrihan', '10A', 65, 40000.00, '2009-01-30', '2023-06-13');

SELECT
    Student_Name,
    Date_of_Birth,
    MONTH(Date_of_Birth) AS Birth_Month,
    DAY(Date_of_Birth) AS Birth_Day,
    CURDATE() AS Today,
    TIMESTAMPDIFF(YEAR, Date_of_Birth, CURDATE()) AS Age
FROM STUDENT;
```

```
+--------------+---------------+-------------+-----------+------------+-----+
| Student_Name | Date_of_Birth | Birth_Month | Birth_Day | Today      | Age |
+--------------+---------------+-------------+-----------+------------+-----+
| Sachith      | 2008-05-15    |           5 |        15 | 2026-09-03 |  18 |
| Shourya      | 2008-11-20    |          11 |        20 | 2026-09-03 |  17 |
| Sparsh       | 2009-02-10    |           2 |        10 | 2026-09-03 |  17 |
| Raghu        | 2008-08-25    |           8 |        25 | 2026-09-03 |  18 |
| Shrihan      | 2009-01-30    |           1 |        30 | 2026-09-03 |  17 |
+--------------+---------------+-------------+-----------+------------+-----+
```

4) Aggregate Functions –

OneCompiler Upgrade AI MySQL Run

queries.sql

```sql
CREATE TABLE STUDENT (
    Student_ID INT,
    Student_Name VARCHAR(50),
    Class VARCHAR(10),
    Marks INT,
    Fee_Paid DECIMAL(10,2),
    Date_of_Birth DATE,
    Admission_Date DATE
);

INSERT INTO STUDENT (Student_ID, Student_Name, Class, Marks, Fee_Paid, Date_of_Birth, Admission_Date) VALUES
(101, 'Sachith', '10A', 85, 50000.00, '2008-05-15', '2023-06-10'),
(102, 'Shourya',  '10A', 92, 50000.00, '2008-11-20', '2023-06-12'),
(103, 'Sparsh',   '10B', 76, 45000.00, '2009-02-10', '2023-06-11'),
(104, 'Raghu',  '10B', 88, 48000.00, '2008-08-25', '2023-06-15'),
(105, 'Shrihan',  '10A', 65, 40000.00, '2009-01-30', '2023-06-13');

SELECT
    COUNT(*) AS Total_Students,
    SUM(Marks) AS Total_Marks,
    AVG(Marks) AS Average_Marks,
    MAX(Marks) AS Highest_Marks,
    MIN(Marks) AS Lowest_Marks
FROM STUDENT;
```

Save

I/O AI Agent 121 ms

STDIN

Input for the program (Optional)

Output:

```
+-----------------+-------------+---------------+---------------+--------------+
| Total_Students | Total_Marks | Average_Marks | Highest_Marks | Lowest_Marks |
+-----------------+-------------+---------------+---------------+--------------+
|               5 |         406 |       81.2000 |            92 |           65 |
+-----------------+-------------+---------------+---------------+--------------+
```